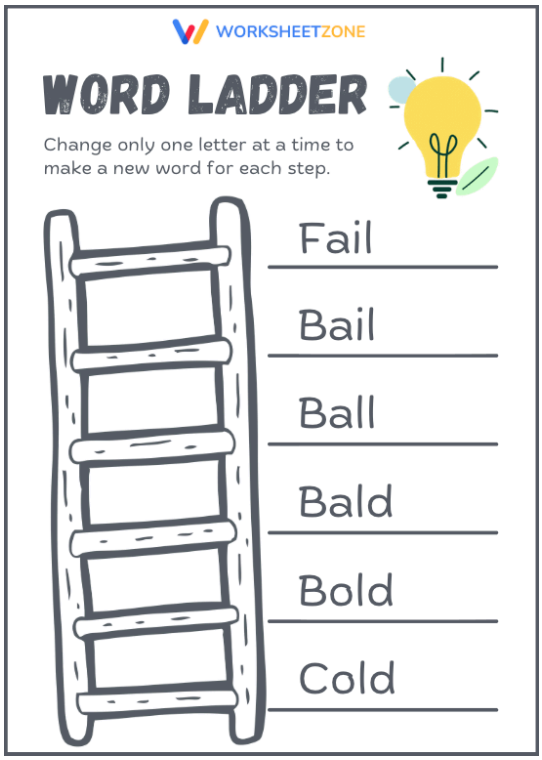
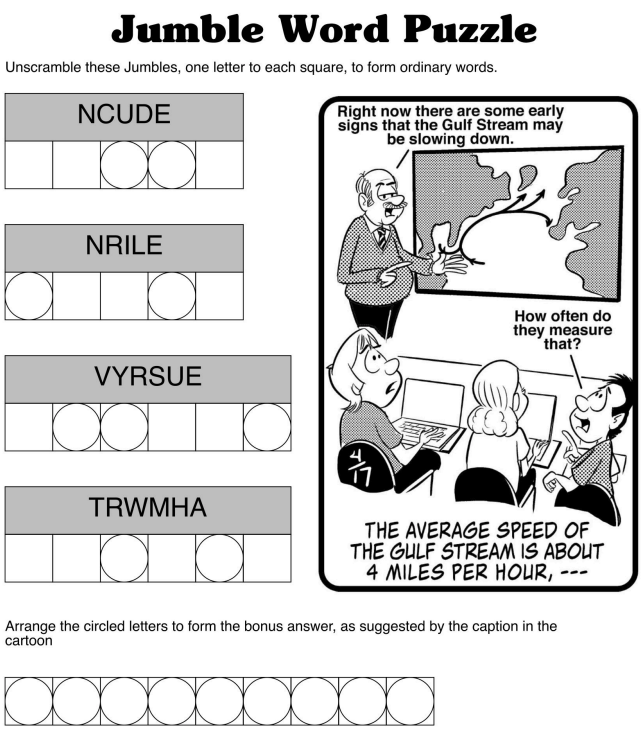
Software Design Document





Word Games
FLEX

Project Manager: Daylin Jackson
Team: Luke Hayden
Zamiyah Hill
Dillon McCants

1 Introduction **2**
1.1 Purpose 2
1.2 Design Goals 2
2 Proposed Software Architecture **4**
2.1 Subsystem Decomposition 4
2.2 Hardware/software mapping 5
2.3 Persistent data management 6
2.4 Access control and security 6
2.5 Global software control 7
3 Subsystem Service **7**

1 Introduction

1.1 Purpose

The purpose of FLEX is to give players a fun, easy-to-use way to practice vocab,spelling, and problem-solving through two classic word games: Jumble and Word Ladder. In Jumble, the player unscrambles a set of letters to form a real word. In Word Ladder, the player changes a starting word into a target word one letter at a time, masking a valid word at each step.

The system is a self-contained Java application that the player runs from a single .jar file with no installation. From the main menu, the player picks a game, reads its instructions, and plays through a series of puzzles. The system checks each answer, gives immediate feedback, keeps track of the score, and uses a life system that ends the game after too many wrong answers. Difficulty ranges from easy to hard, and harder levels use more words and longer puzzles,

To keep players engaged, especially children, each new game uses a randomized theme (such as Christmas, Halloween, Thanksgiving, or one of the four seasons) with matching illustrations and music. The project is meant both as a casual pastime and as a learning tool for elementary school students, so it is designed to be friendly and approachable for players of any age or reading level.

1.2 Design Goals

End-User Criteria

Usability: Both games must be simple enough for anyone to understand, including young children. Menus should be clear, and instructions for each game must be available at any time.
Learnability: Easier difficulties will use common words, so players with a limited vocabulary can still succeed. Players should be able to start playing without outside help.
Engagement: Fun designs and music for players to see and listen to, varying difficulty levels which challenge the brain, and two different games to choose from.

Performance Criteria

Response time: A game should start immediately with no noticeable loading time.

Feedback: The player should know whether an answer is correct the moment it is submitted, with no input lag or sluggish behavior.

Dependability Criteria

Robustness: Invalid input (empty answers, numbers, symbols, wrong word length, or Word Ladder move that changes more than one letter) must be handled gracefully without crashing the game.
Reliability: If an error occurs, the player should be able to keep playing quickly or not notice the error at all. Because no data is saved between sessions, data loss is not a concern.

Maintenance Criteria

Modifiability: Although the games are not expected to need updates, the design should make changes easy, such as adding new words, puzzles, themes, or difficulty levels without rewriting game logic.
Extensibility: Jumble and Word Ladder share a common game structure, so a new word game could be added later with minimal changes to the rest of the system.
Readability: Code will be organized and documented so the lead programmer, Dillon, or anyone on the team, can maintain it. All changes are tracked through commits on Github.

Cost and Deployment Criteria

Portability: The system must run on any computer that can run Java executables.
Simple deployment: The games are packaged as a single .jar file hosted on Github, so no installation is required.
Development cost: The system will be built in Java using free tools and released under the MIT License, with no licensing costs.

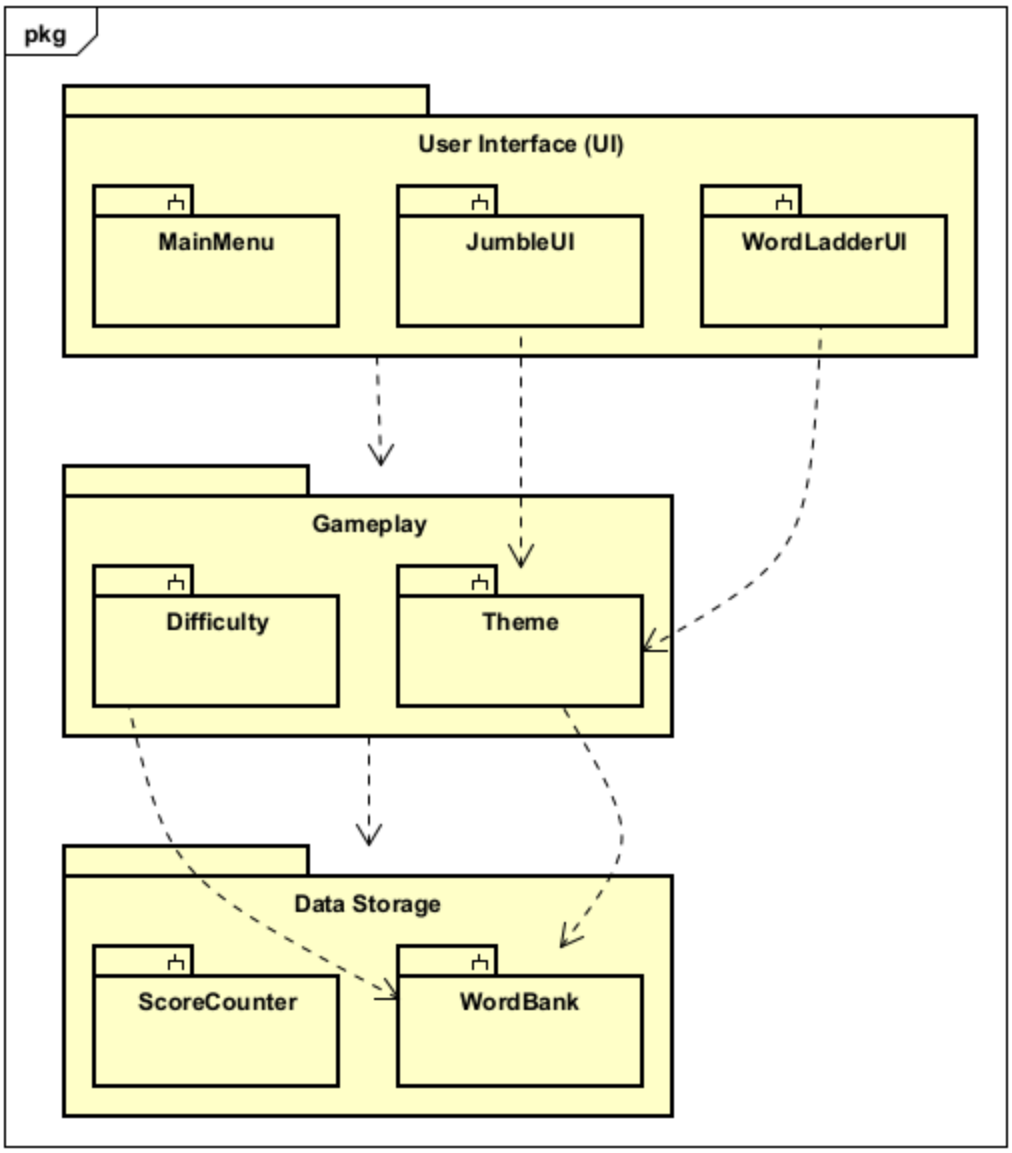
Trade-offs

Simplicity vs. Features: When a feature would make the game harder to understand, simplicity for the player takes priority.
Engagement vs. Performance: Themes, images, and music must not slow down game startup or answer checking. If they would, assets will be kept lightweight or loaded in the background.

2 Proposed Software Architecture

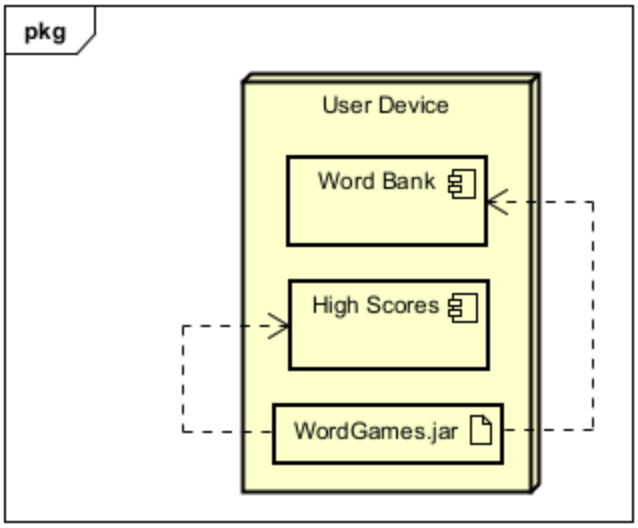
2.1 Subsystem Decomposition

- **Gameplay** (handles background elements)
    - **Difficulty** (uses different words depending on difficulty)
    - **Theme** (sets background and music based on selected theme)
- **Data Storage** (handles data acquisition and requests)
    - **Score Counter** (saves the user's score in a text file)
    - **Word Bank** (contains all words available to be used for each difficulty)
- **User Interface (UI)** (handles everything that the user sees and interacts with)
    - **Main Menu** (what the user sees upon starting the application; lets the user pick difficulty, the game they want to play, and theme)
    - **Word Ladder UI** (what the user interacts with to play Word Ladder)
    - **Jumble UI** (what the user interacts with to play Jumble)



2.2 Hardware/software mapping

The Java executable, high score file, and word bank are all contained in a single, downloadable folder. Everything is self-contained; nothing outside of the folder will be accessed.



2.3 Persistent data management

High scores for both games will be saved in a text file. A word bank will also be stored in a text file. These text files will be stored in a “Saves” folder located next to the application executable, all inside of a “Word Games” folder. High scores will be written to the text file in the format GAME_NAME:SCORE (e.g. JUMBLE:123). The word bank file will be in the format DIFFICULTY:WORD1,WORD2,WORD3,... (e.g. EASY:DOG,CAR,BIG,…). To load the high scores, the line that contains the desired game name will be split by the colon to isolate the score. To load the word bank, the line that contains the current difficulty will be split by the colon to isolate the words. The isolated words will then be separated by the commas to make a list.

2.4 Access control and security

Our word games system is designed to be played without requiring the user to create accounts or input any private information. The users should be able to access both Word Ladder and Jumble directly through the application without requiring them to input information that is not saved locally. Because of this, authentication is not a requirement as we are not needing to save any sort of personal information that could be at risk, allowing us to continue forward without an encryption system.

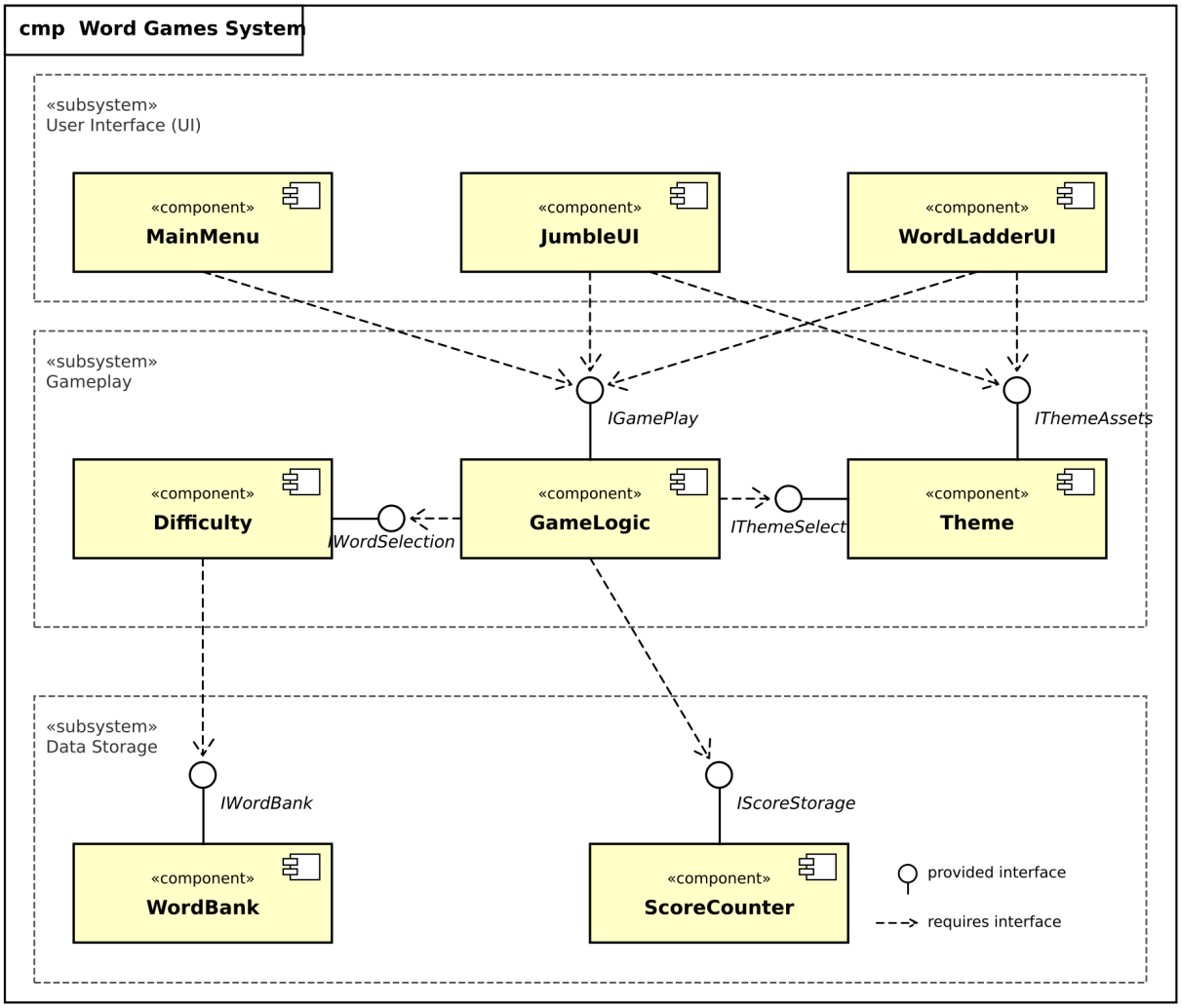
The application will be able to be accessed by anyone who installs it and is able to run the lightweight application. We will use the system to validate inputs or answers from the users allowing us to ensure that our game is working properly and the answers users input are checked for validity. Any data the game has will be handled with the applications design. We will not have an encryption key system because no sensitive information or data will be stored.

2.5 Global software control

The high scores and word bank are both read and loaded upon starting the application. The user theoretically could change the values at any time, but only the data that was present when the application started will be used. Therefore, there will be no synchronization issues.

3 Subsystem Service

The FLEX system is divided into three subsystems: User Interface, Gameplay, and Data Storage. Each subsystem is made up of components that provide services to the others through interfaces. The User Interface components depend on Gameplay, and Gameplay depends on Data Storage, so each layer only talks to the layer directly below it. This keeps the system easy to maintain, since a change to how words or scores are stored will not affect the screens the player sees.



User interface subsystem

- **MainMenu**: The first screen the player sees when the application starts. It lets the player choose a game(Jumble or Word Ladder), a difficulty, and a theme, and it can display instructions and high scores. It requires the IGameplay interface to start a new game with the player's choices.
- **JumbleUI**: Displays the scrambled word, the answer box, the player's score, and remaining lives. It sends the player's answers to GameLogic and shows whether each answer was correct. It requires IGamePlay and IThemeAssets.
- **WordLadderUI**: Displays the starting word, the target word, and each step of theladder the player has built. It sends each new word to GameLogic and shows the result. It requires IGamePlay and IThemeAssets.

Gameplay Subsystem

- **GameLogic**: The core of the system. It starts and ends game sessions, checks answers against the rules of Jumble and Word Ladder, rejects invalid input, updates the scores and tracks lives. It provides **IGamePlay**(startGame, submitAnswer,getScore,getLives, endGame) to the UI components. It requires IWordSelection, IThemeSelect, and IScoreStorage.
- **Difficulty**:Chooses which words are used based on the selected difficulty(Easy, Medium, or Hard). It provides **IWordSelection**(getWordList, getRandomTheme) to GameLogic and IThemeSelect(setTheme, getRandomTheme) to GameLogic and IThemeAsseers(getBackground,playMusic,stopMusic) to the game screens.
- **Theme**: Sets the background,illustrations, and music for the selected or randomized theme. It provides IThemeSelect (setTheme, getRandomTheme) to GameLogic and IThemeAssets(getBackground, playMusic,stopMusic, stopMusic) to the game screens.

Data Storage Subsystem

- **WordBank**: Reads the word bank text file when the application starts. Each line is split by the colon to find the difficulty, and the words are split by commas into a list. It provides **IWordBank**(loadWords, getWords(difficulty)) to Difficulty.
- **ScoreCounter**: Reads and writes the high score text file in the format GAME_NAME:SCORE. It provides **IScoreStorage**(loadHighScore(game), saveHighScore(game,score)) to GameLogic,which saves a new high score when a game ends.